Deep Learning in Computer Vision· Lecture 9 of 9

YOLO: You Only Look Once

Real-time detection in one pass and course wrap-up

Where We Are, and Roadmap

- **Lecture 8**: two-stage detectors are accurate but slow (propose, then classify).
    - Today: **YOLO**, which predicts everything in **one pass**.
        - the YOLOv1 idea: grid, predictions, confidence.

Glossary: Terms Used in This Lecture

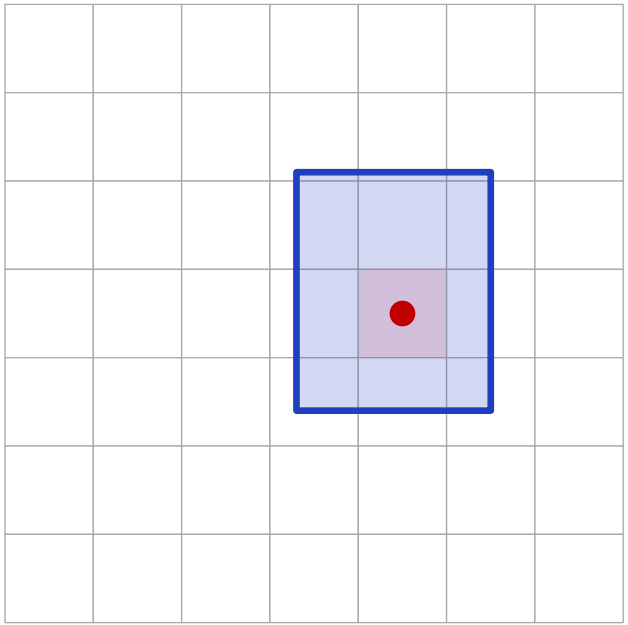
Term	Meaning
Grid cell	One square of the S × S grid laid over the image
Bounding box	(x, y, w, h) rectangle predicted around an object
Confidence / objectness	Score that a box contains an object (and fits it well)
Class probability	Probability of each class, given that an object is present
Tensor	A multi-dimensional array of numbers (here the network output)
Backbone / neck / head	Feature extractor / feature mixer / final predictor
FPS	Frames per second: speed of the detector

Motivation:

	Two-stage (Faster R-CNN)	YOLO (2016)
Steps	Proposals → crop features → classify → refine	One network, one forward pass
View	Looks at each region separately	Looks at the whole image at once
Training	Several parts / stages	End to end with one loss
Speed	≈ 7 fps (Faster R-CNN)	45 fps (Fast YOLO: 155 fps) **real-time** detection

- **Name**: "**You Only Look Once**": the image passes through the network only once.
- **YOLO** (Redmon et al., 2016): treat detection as **one regression problem** from image pixels straight to box coordinates and class probabilities.

Core Idea: A Grid of Responsibilities



7×7 grid; red dot = object centre

- Divide the image into an **S × S grid** (S = 7 in YOLOv1, yielding $7 \times 7 = 49$ grid cells).
- The cell that contains an object's **center** (red cell) is **responsible** for detecting that object, even if the object extends over many cells.
- Every cell predicts **B boxes** (B = 2 in YOLOv1, each with a confidence) and **C class probabilities** (e.g., in PASCAL VOC, $C = 20$ classes like person, car, dog, chair).
- No proposals, no second stage: all cells predict **simultaneously**.

What Each Cell Predicts

- For each of **B = 2** boxes: **x, y, w, h, confidence** (5 numbers).
- Once per cell: **C = 20** class probabilities (Pascal VOC classes). The two boxes of a cell **share** these probabilities, so a cell predicts **one class**.

Symbol / term	What it means
S	grid size per side (7)
B	boxes predicted per cell (2)
C	number of classes (20)
x, y	box centre, relative to the cell (0-1)
w, h	box size, relative to the whole image (0-1)
confidence	how sure the box holds an object and how well it fits

The Output Tensor: 7 × 7 × 30

$$S \times S \times (B \times 5 + C) = 7 \times 7 \times (2 \times 5 + 20) = 7 \times 7 \times 30$$

Per box (×2)

x, y, w, h, confidence = **5 numbers** each → **10** per cell

Per cell (×1)

20 class probabilities → 10 + 20 = **30** numbers per cell

Whole image

7·7 = **49 cells** → **1,470 numbers**; up to **98 boxes** (49 × 2)

A "tensor" is just this 3-D block of numbers: 7 rows × 7 columns × 30 channels.

Decoding a Box: Worked Example

- $S = 7$, image 448×448 → each cell is **64×64 px**. Cell **column 4, row 3** predicts $x = 0.5$, $y = 0.5$ (inside the cell), $w = 0.3$, $h = 0.5$ (fractions of the image).

Quantity	Calculation	Result
Center x	(4 + x) × 64 = (4 + 0.5) × 64	288 px
Center y	(3 + y) × 64 = (3 + 0.5) × 64	224 px
Width	w × 448 = 0.3 × 448	134 px
Height	h × 448 = 0.5 × 448	224 px

- Why relative values? All outputs stay in **0-1**, which makes the regression **stable and scale-independent**.

Confidence: Object and Fit

confidence = Pr(Object) × IoU(pred, truth)

Symbol / term	What it means
Pr(Object)	1 if an object's center is in the cell, else 0 (training target)
IoU(pred, truth)	overlap of the predicted box with the real box.
Target	no object → confidence 0; object → the IoU itself
Meaning	high only if an object is present and the box fits it well

- Example: object present and predicted box overlaps the truth with IoU 0.8 → target confidence **0.8**; an empty cell → **0**.

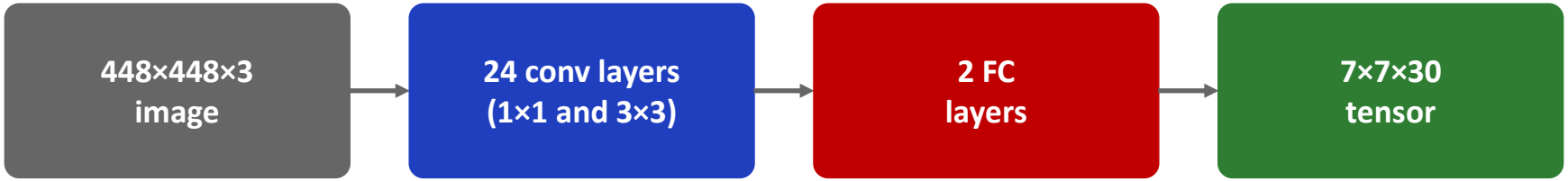
"is there an object, and is the box tight?"

Class-Specific Score at Test Time

$$\text{score}_c = \Pr(\text{Class}_c \mid \text{Object}) \times \Pr(\text{Object}) \times \text{IoU}$$

- **Pr(Class_c | Object)**: probability of class c **given** an object is there (cell's class output).
- Multiply by the box confidence to get one **score per class per box**; it encodes both **which class** and **how well the box fits**.
- **Example**: box confidence 0.8; class probabilities dog 0.9, cat 0.1 → dog = 0.9 × 0.8 = **0.72**; cat = 0.1 × 0.8 = **0.08**.
- Boxes with low scores are dropped; the rest go to **NMS**.

The YOLOv1 Network



- Inspired by **GoogLeNet 1×1 convs** reduce channels before costly 3×3 convs.
- The first **20 conv layers are pretrained on ImageNet** at 224×224 (≈ 88% top-5), then extended with 4 conv + 2 FC layers and trained for detection at **448×448** (detection needs finer detail).
- **Leaky ReLU** activations: $f(x) = x$ for $x > 0$, $0.1 \times x$ otherwise (avoids "dead" neurons).
- The last FC layer outputs 7×7×30 = **1,470 values**.

YOLOv1 Layer Sizes

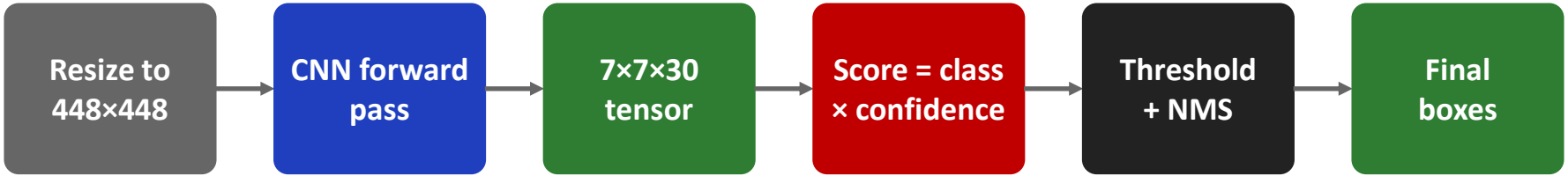
Stage	Operations	Output
Input		448×448×3
1	conv 7×7×64, stride 2 → max-pool 2×2	112×112×64
2	conv 3×3×192 → max-pool	56×56×192
3	1×1×128, 3×3×256, 1×1×256, 3×3×512 → max-pool	28×28×512
4	4 × (1×1×256, 3×3×512), 1×1×512, 3×3×1024 → max-pool	14×14×1024
5	2 × (1×1×512, 3×3×1024), 3×3×1024, 3×3×1024 (stride 2)	7×7×1024
6	2 × conv 3×3×1024	7×7×1024
Head	FC 4096 → FC 1470 → reshape	7×7×30

The first FC layer alone has 7·7·1024·4096 ≈ 205M weights: later versions drop FC layers entirely.

Which Box Is "Responsible"?

- A cell predicts **B = 2** boxes but a real object needs only one.
- During training the box with the **higher IoU** with the ground truth is made responsible (1 obj = 1); the other gets no-obj.
- This makes the boxes **specialize**: one tends to prefer wide shapes, another tall, and so on, improving recall.
- Training details: pretrained backbone, SGD with momentum, ≈ 135 epochs, **dropout** and **data augmentation** (random scaling, translation, colour changes) against overfitting.

Inference Pipeline



- A **single forward pass** produces up to 98 candidate boxes; each gets class-specific scores.
- **Threshold** (e.g. 0.2) removes weak boxes; **NMS** removes duplicates.
- The network sees the **entire image**, so it uses **global context (i.e., fewer background mistakes)**:
    - it makes **less than half the background mistakes** of Fast R-CNN.

YOLOv1 Results: Speed vs Accuracy

Detector (Pascal VOC 2007)	mAP (%)	FPS
Fast YOLO	52.7	155
YOLO (v1)	63.4	45
Faster R-CNN (ZF)	62.1	18
Faster R-CNN (VGG-16)	73.2	7
Fast R-CNN	70.0	0.5

- **Real-time (≥ 30 fps) detection was new**: YOLO gave up some accuracy for **6× the speed** of Faster R-CNN (VGG-16).
- **mAP** = mean average precision; **FPS** = frames per second. Figures from the YOLO paper.

Limits of YOLOv1

- **Spatial constraint**: each cell has **2 boxes and 1 class**, so **groups of small objects** (a flock of birds) in one cell cannot all be detected.
- **Coarse features**: the 7×7 output grid (heavily downsampled) is poor for small objects.
- **Unusual aspect ratios**: boxes are learned from training data and generalize badly to new shapes.
- **Localization errors**

- These limits motivated **YOLOv2** and later versions.

The YOLO Family: Timeline

Version	Year	Main additions
v1	2016	Single-pass grid detector (Redmon et al.)
v2 / 9000	2016-17	BN, anchors, k-means priors, multi-scale training
v3	2018	Darknet-53 (residual), 3-scale heads, multi-label classes
v4	2020	CSPDarknet53 backbone, PANet neck, Mosaic augmentation, CIoU loss
v5	2020	PyTorch implementation (Ultralytics): easy training and export
v6 / v7	2022	Industrial / efficiency-focused redesigns (Meituan; Wang et al.)
v8	2023	Anchor-free, decoupled head; detection, segmentation, pose
v9 – v11	2024	Better gradient flow / efficiency; NMS-free training (v10)
Yolo26, Yolo27	2026, 2027	…

New releases keep appearing: check the Ultralytics documentations.

Model Sizes: Speed vs Accuracy (YOLOv8)

Model	Params	COCO mAP50-95
YOLOv8n (nano)	3.2 M	37.3
YOLOv8s (small)	11.2 M	44.9
YOLOv8m (medium)	25.9 M	50.2
YOLOv8l (large)	43.7 M	52.9
YOLOv8x (extra large)	68.2 M	53.9

- **Same design, different width and depth.**
- Bigger → more accurate, slower, more memory.
- **mAP50-95** = COCO mAP averaged over IoU 0.50-0.95 (Lecture 8).
- **Pick** n/s for phones and edge devices, m/l for GPUs, x for best accuracy.
- Values as reported by Ultralytics (640 px); check the docs.

Choosing the Right Task and Model

I need to know…	Task	Typical model
What is in the image	Classification	ResNet, EfficientNet, ViT
What and where (boxes), real time	Detection	YOLO
What and where, max accuracy	Detection	Faster R-CNN, large transformer detectors
Exact region of each class	Semantic segmentation	U-Net, DeepLab, SegFormer
Separate mask per object	Instance segmentation	Mask R-CNN, YOLO-seg

Recurring Ideas Across the DL in computer Vision

- **Learn features from data, end to end** with backprop instead of hand-crafting them.
- **Skip connections** (ResNet, U-Net, FPN, transformers): keep detail and gradients flowing.
- **Transfer learning**: reuse ImageNet / COCO-pretrained backbones.
- **Class imbalance** appears everywhere (rare disease, background pixels, empty cells): fix with weighting, Dice, focal loss, metrics beyond accuracy.
- **Overlap metrics**: precision / recall → IoU → AP / mAP.
- **Beyond this course**: video, 3D vision, self-supervised and **foundation models** (CLIP, SAM).

Check Yourself: Questions and Answers

Question	Answer
Output size for S = 7, B = 2, C = 20?	7 × 7 × 30 = 1,470 numbers.
Which cell detects an object?	The cell containing the object's centre.

- **Fine-tune or train from scratch with say 500 images??**
  - **In practice**: fine-tune a pretrained model on YOLO-format data; judge by **mAP**, PR curves and error analysis.

Questions